Microservices Interview Questions

1. What is Coupling?

Dependency: Dependency is an essential functionality, library, component, service or code that essential for a different part of the software system to work.

Coupling: Is a degree of interdependence of software modules, components or services.
High Coupling: Too much dependency

2. Type of Couplings?

Content Coupling: Once class can access the private members of another class
Java and C#.Net doesn't allow to access private members.
C++ will allow to access private numbers.
Common Coupling: Two classes access the same shared data i.e., global variables and static properties
Control Coupling: When a function controls the flow of another function
Routine Coupling: When one function calls another function without passing any parameters.
Data Coupling: When two systems share the same database
Type Use Coupling: When a member or property of class B is of type class A
Stamp Coupling: When in Class B a method has a parameter of type class A
Import Coupling: When a library is imported into another program.
External Coupling: When we communicating with external programs.

3. What is Cohesion?

What is better? High Cohesion or Low Cohesion?

Is a measure of the degree to which the elements of a module, software or code are related.
Is a degree to which all elements of a module, software or code are directed towards performing a single task.

High Cohesion is GOOD!! -> low cohesion are helper classes.

4. What is Microservice?

- Microservices are an architectural and organizational approach to software development where software is composed of small independent services that communicate with each other in loosely coupled manner.
- Microservices try to solve the problems of Monolithic Applications
- With Monolithic architectures, all processes are tightly coupled and run as single service.

5. Pros and Cons of Monolithic Applications

Pros:

Simplicity: Monolithic Applications are easier to build, test and deploy.

Cross-cutting concerns: One piece of code can handle monitoring, logging, security etc.

Performance: Different classes or functions communicate directly and share the same memory. So, the applications runs faster compared to microservices where services communicate through a network.

Cons:

Reliability: A fatal error in any part of the application will crash the entire system.

Updates and Deployments: Even for a small change the entire application has to be re-deployed.

Technology Stack: The entire application has to be developed with one single technology stack.

6. What are alternatives of Monolith Apps

- Single codebase monolith apps.
- Component based monolith apps
- Service-Oriented Architecture
- Microservices Architecture.

7. Benefits of Microservices

- Microservices can be deployed independently
- Microservices can be scaled out independently
- Microservices offer better fault tolerance
- A smaller code base allows new team members learn code easier and quicker
- Different microservices can be built with different technology stacks
- Microservices can communicate asynchronously – reducing coupling (with help of message brokers like Kafka or JMS).

8. Drawbacks of Microservices

- Microservices-based systems are mode complex to design and build
- Requires changes in how teams and organizations work
  - More teams to manage
  - Often more developers are required.
  - Teams have to collaborate more efficiently
- Often more expensive to build
- Diagnosing problems become difficult.
- Testing becomes more challenging.

9. How do microservices relate to the business?

- Business prefers to use cloud i.e., AWS as opposed to on-prem infrastructure to avoid a large Capex.
- Micro services are a great fit for cloud.
- Micro services enable businesses to better implement agile.
- Micro services allow businesses to out-source part of their development effort, and get to market quicker.

10. Name some design patterns and tools we use in developing microservices

- Event Streaming
- Message Broker
- Containers & Dockers
- No-SQL Database
- Restful API
- API Gateway

11. What is Blast Radius?

- Blast Radius is the degree to which the entire system is affected if a micro service fails or shuts down.
- In monolith application the blast radius is 100%

  If part of the application encounters a fatal error

  If the database becomes unavailable (due to network misconfiguration, overloading) or breaks.
- In the micro services architecture, we aim at reducing the blast radius with Patterns of Resilience.
  - Independent database for each microservices
  - Timeouts
  - Backoff strategy(retries)
  - Circuit breaker pattern
  - Bulkhead pattern
  - Fallback pattern
  - Asynchronous communication (no point-to-point calls)

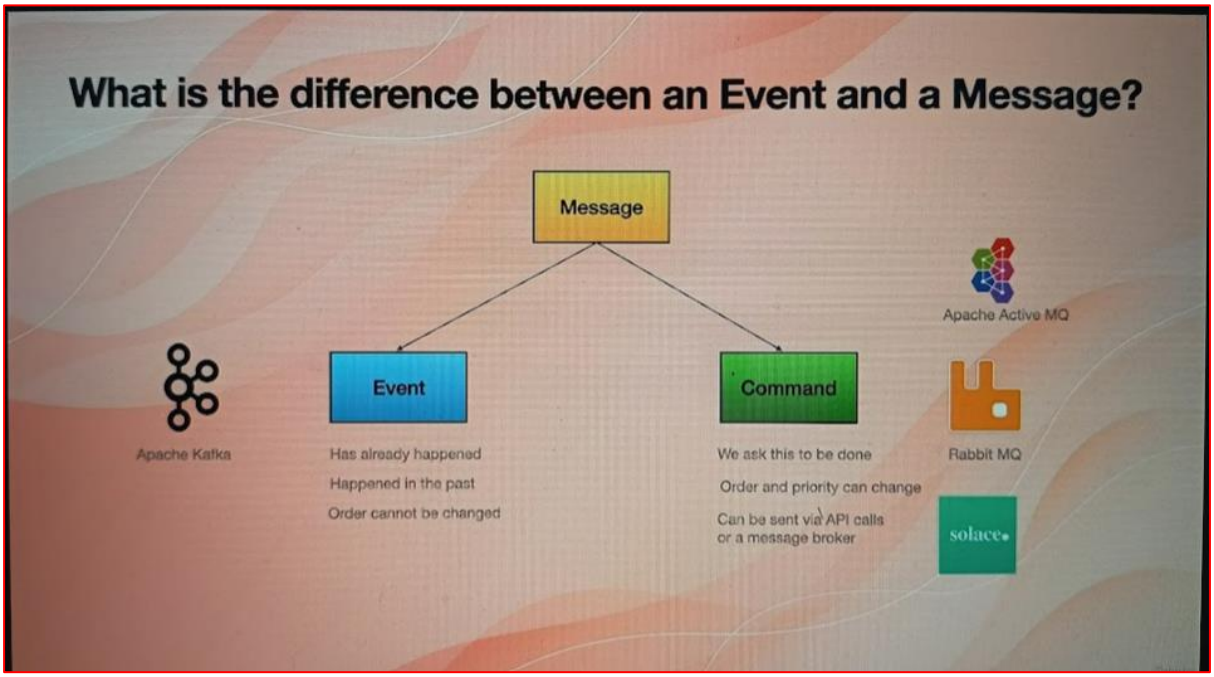
12. What is Circuit Breaker Pattern?

- We set threshold for failing API calls i.e., 3
- If failing API calls exceed the threshold, we reject the API calls
- Circuit breakers are micro services themselves and act as Proxy
- Circuit breakers can have three states: Open, Closed, Half-Open

13. Name some ways or architectural styles of building microservices?

- **Single-tenant micro services:** One micro service on one virtual server
  - Good for migrating monolith to micro services
- **Containerised micro services:** Each micro service runs as Docker container
- **Serverless:** Small micro services that run without a server in the cloud Amazon Lambda in AWS.
- Spring Boot in Java

14. What is the difference between an Event and a Message?



15. What is a Distributed Transaction?

- In Microservices-based architecture each service has its own database
- We need to make sure transactions (insert, update, delete) are ATOMIC across all services
- Managing transactions across multiple services is Distributed Transactions.

How is Distributed Transactions Handled?

- Two-Phase Commit Transactions(2PC) (not recommended)
- Saga Pattern

16. What is Two-Phase Commit Pattern(2PC)

- Widely used in databases
- Sometimes useful in microservices (only a few services in the system)
- It has a prepare phase and a Commit phase

- **Microservices are asked to prepare for change** (Crate a record in the database and set its status field or column to 0(0 means not stable))
- **Microservices are asked make the final and actual change** (Set the status field or column to 1(finalised)).

➢ A Coordinator microservices is needed to maintain the life-cycle of the transaction.

17. What is Saga Pattern?

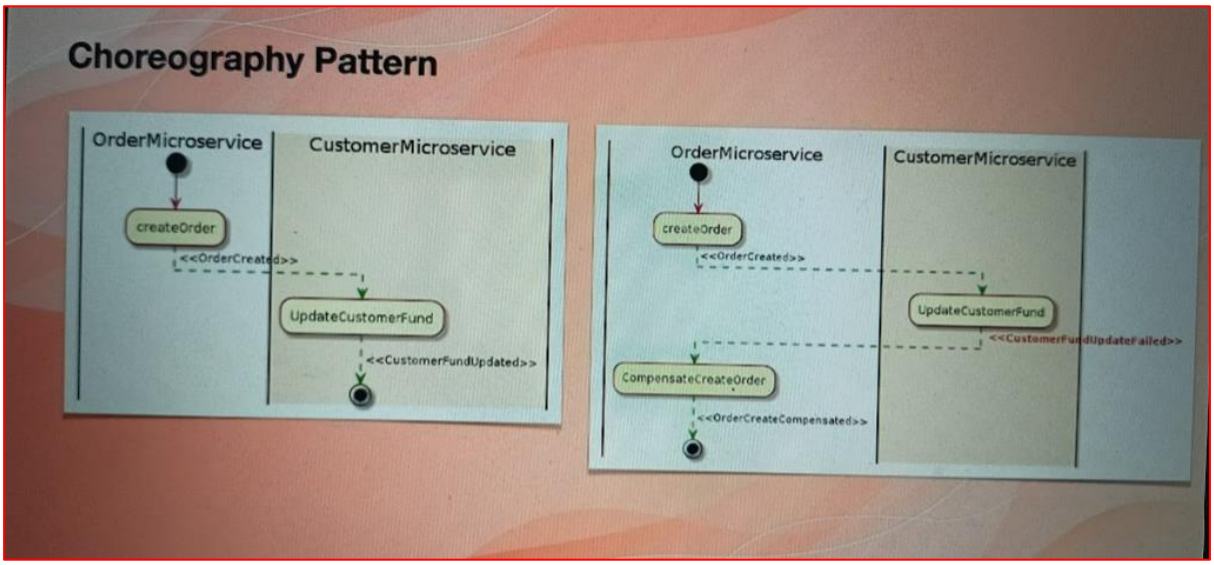
➢ Asynchronous

➢ Each micro service handles its own transaction

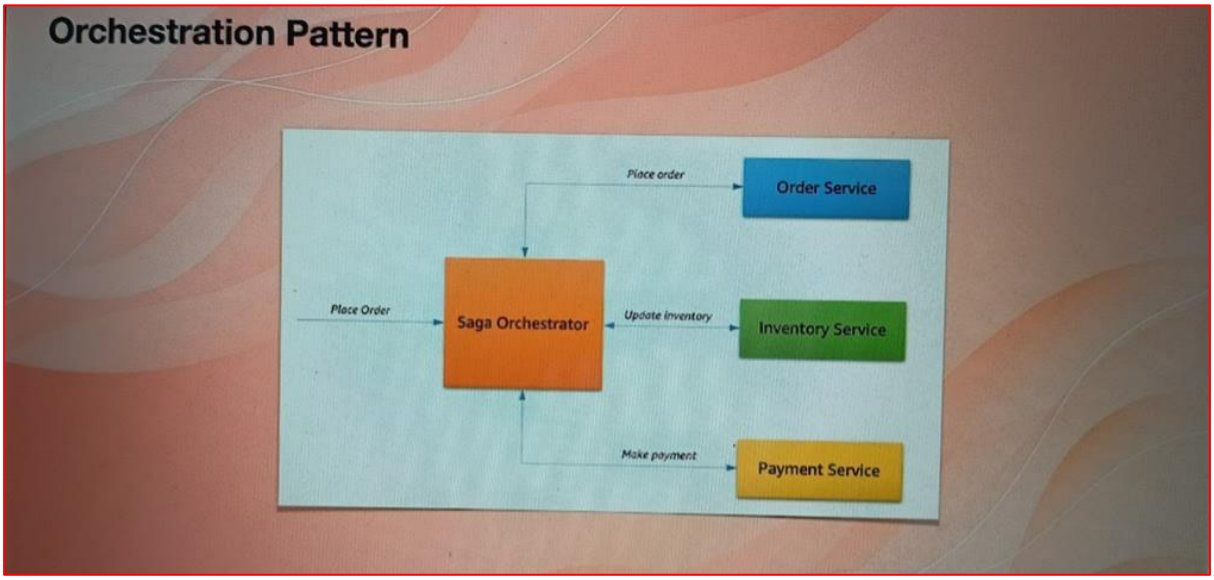
➢ There are two-patterns

- **Choreography patterns**
  If something goes wrong, micro services have to tell the up-stream(previous) micro services to Roll Back the transaction
- **Orchestration patterns**
  A central micro service delivers the messages for roll-backs.

18. What is Choreography Pattern?



19. What is Orchestration Pattern?



20. What are the differences between Monolithic, SOA and Microservices

Monolith	SOA	Microservice
Application is made of a single code base	Tries to break down the monolith app to smaller components	Microservices are more fine-grained and smaller compared to SOA
Application runs as a single process and entire code shares the same memory	Services communicate via Synchronous API calls (SOAP or REST)	Microservices communicate via Rest APIs, Events and Messages
All functions in the code share the same database	In SOA often services share the same database although not a best practice	Each service has its dedicated database

21. What is Bounded Context?

- Bounded Context is a concept in Domain Driven Design (DDD)
- DDD is a framework of analysing and modelling large problems, models and teams
- The entire problem is a Domain i.e., Drone Delivery System

- A Domain Model is the representation of a real thing in the world i.e., Drone, User, Package.
- A bounded context is simply the boundary within a domain where a particular domain model applies.
- Normally one microservice represents one bounded context.

22. Explain how independent microservices communicate with each other?

- Point-to-Point & Synchronous API calls (not recommended)
- Responding to an Event
- Responding to Message

23. Explain CDC ?

- CDC stands for Consumer Defined Contract.
- It's a kind of test to ensure that constant changes in micro services will not break the dependent microservices
- We have Provider (API) and Consumer (that uses the API)
- The consumer has to write the test for the contract between it and the Provider. In the pre-stage of deployment, Consumer will run the tests. Failing to pass the contract, deployment is not taken further.
- The Provider also runs the consumer test on its side before deploying it to make sure that its changes won't affect the consumers to whom it is providing the service.

24. What are Client Certificates?

- Client Certificate is a method for authenticating the API user.
- A Client Certificate file (public & private key pair) is used
- No encryption of data is done by Client Certificate
- When API caller(client)makes an API call (connects to server), the server validates the Client Certificate
- This process is called TLS-hand shake.

25. What is Semantic Monitoring?

- Monitoring microservices is crucial.
- Server layer monitoring
    - Is my microservice working as expected?
    - Achieved via Health Endpoints(/health), Telemetry (i.e., Prometheus & Grafana) etc
- Semantic monitoring
    - Approaches microservice monitoring from the business transaction, or semantic, perspective.
    - How well the transaction performs for the business and the customers who use it
    - Achieved via Functional Testing.

26. What is Continuous Monitoring?

- Continuous Monitoring (CM) is also known as Continuous Control Monitoring (CCM)
- CM is an automated process that allows engineers to detect compliance and security threats in their software development lifecycle and infrastructure.
- Unlike traditional manual & periodical checks, CM helps to identify and track key risks in real time because it uses automation.

27. Explain OAuth?

- OAuth stands for Open Authentication
- Users and Credentials are stored in an authentication server i.e., Google
- Credentials are made of Scope and Claim
    - **Scope:** What information can be accessed by the user? i.e., Full Name
    - **Claim:** A set of key-value pairs

28. Why OAuth suits Microservices?

- User authenticates against an auth server ONCE
- Using the same Authentication Token all microservices that use the same Auth server can be accessed (if they allow)

29. Explain Idempotence and its usage?

- Idempotence is a concept: produce same output for same input
- It is used to make microservices more resilient
- Some microservices are triggered by receiving messages from a message broker i.e., ActiveMQ
- Message brokers may send a message more than once
- Microservices must be programmed to handle duplicate messages
- Handling duplicates messages is called Idempotence.
- This is done by
    - Every message must have a unique identifier
    - Microservices stores the messages in local database

30. What is service discovery?

- Service Discovery is used to find where a service is(IP & DNS name)
- This is needed because in the cloud i.e., AWS Services have dynamic Ips or DNS name
- Service Instances (i.e., virtual machine or container) register or write their IP or DNS name in a service registry.
- Service Registry is a Key-Value pair storage. Service Name, IP.

Client-Side Service Discovery

- The client service is responsible for discovering the network location of the services and load balance across them.
- Services register their IP when they start up
- IP of a service is removed using a Heart Beat mechanism
- No Load Balancer will be required.

Server-Side Service Discovery

- The client connects to Service Registry via a Load Balancer
- The load balancer queries the Service Registry
- The load balancer routes the traffic to target micro service

Ex: etcd : A highly available, distributed service discovery system. Used by Kubernetes.

Hashicorp consul: Offers fast service discovery, load balancing and APIs for registering and de-registering services

Apache Zookeeper: Used to coordinate distributed systems. It was built for Hadoop. Commonly used alongside Apache Kafka.

31. Service Registration Patterns?

Self-Registration: Services register and de-register themselves
Third-Party Registration: Another system or microservice does the registration and de-registration.

32. What is Side Car Pattern?

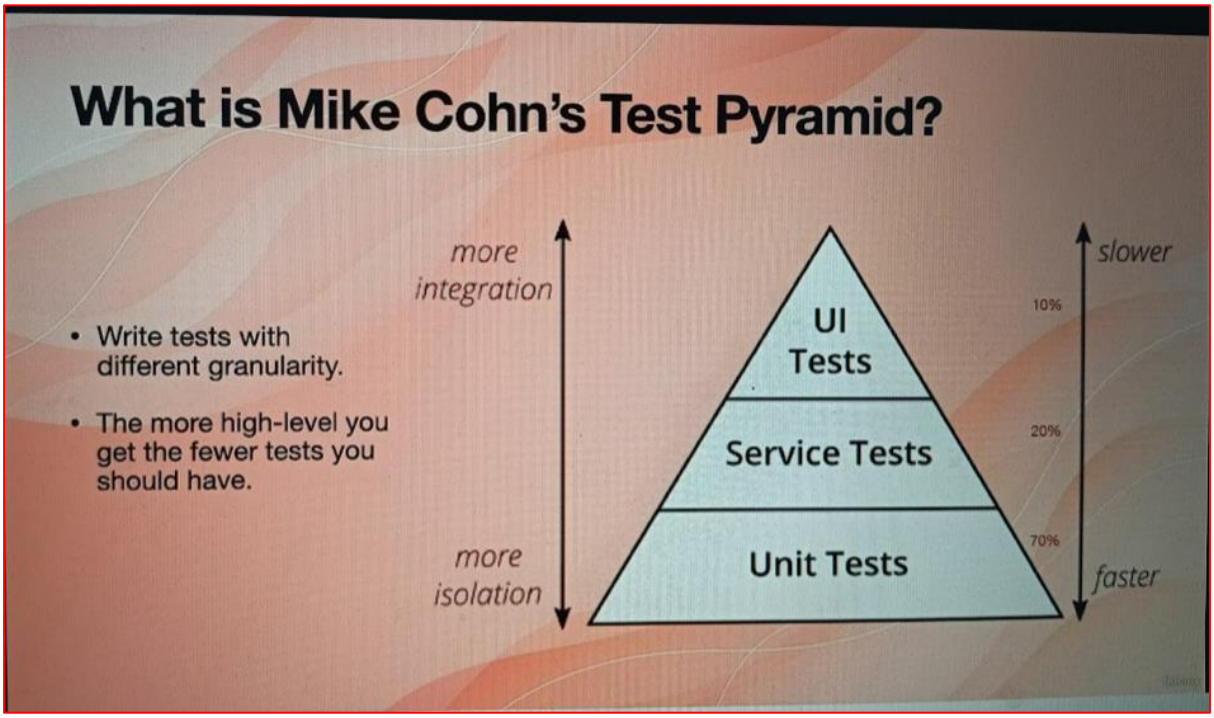
Definition: Deploying components of an application or service into a separate process or container to provide isolation and encapsulation.

Core microservice: mail functionality
Side car Microservice: Logging, Configuration, Proxy to remote service, etc.,

33. What are the types of tests in microservices?

Functional Testing: Is the overall system working?
Load Testing: Does the service scale when the load goes up?
Resilience Testing: How the application reacts to infrastructure failure.

34. What is Mike Cohn's Test Pyramid?



35. Explain Container in Microservices?

- A ***container*** is a building of an application (i.e., micro service) and all its dependencies as a package, that allows it to be deployed easily and consistently regardless of environment.
- For example, a software that only runs on Linux and uses MySQL can be deployed to windows server without having to install MySQL.
- Containers use Virtualisation features of the host operating system.
- Containers allow us to deploy microservices to various environments. Also, services can be built with different technologies but run side by side.

36. What is the main role of docker in micro service?

Create Image: Packages the application and all its dependencies to a binary file called Docker Image.

Run Containers: A running instance of a Docker Image is a Docker Container

Provides Networking: Containers can connect to each other i.e.; Website container connects to MySQL container.

37. How can you deploy containers?

- Deploy to servers that have Docker on them (not recommended -local development)
- Deploy to server-less container services i.e., AWS Fargate
- Deploy to a container orchestration service i.e. Kubernetes. They provide networking, monitoring, logging and access control.

38. What is Restful API?

An API, or application programming interface, is a set of rules that define how applications or devices can connect and communicate with each other.

Uniform Interface: All API requests for the same resource should look the same, no matter where the request comes from.

Client-server decoupling: In REST API design, client and server applications must be completely independent of each other.

Statelessness: REST APIs are stateless, meaning that each request needs to include all the information necessary for processing it.

Cache ability: When possible, resources should be cacheable on the client or server side.

Layered system architecture: In REST APIs, the calls and responses go through different layers. Don't assume that the client and server applications connect directly to each other.

39. What are the ways of testing the security of micro services?

DAST: Dynamic Application Security Testing → Black Box.

- Tests the security of the end-to-end application
- Is done by injecting malicious data or input. I.e., SQL Injection

SAST: Static Application Security Testing → White Box Testing

- Finds the security issues by scanning and reviewing the source code.

IAST: Interactive Application Security Testing

- Suits more modern applications. I.e., Mobile Apps
- Combines both DAST and SAST approaches
- Places an engine within the application to analyse the app in real time, development and QA

RASP: Run-time Application Security Protocol. It allows the applications to perform continuous security checks on itself and terminate the suspicious sessions.

40. Explain the Scaling Cube?

X-Axis: Add more servers

- AWS EC2 & Autoscaling
- Group is an example

Y-Axis: Split the system by functions: Microservices

Z-Axis: Portioning

Example: Handle customers of each country by different servers closer to them.

41.What is Data Offloading?

- Microservices normally run in virtual machines i.e., containers
- Microservices may come and go because virtual machines are not stable
- Microservices must not store data locally because if they go data will go with them.
- Microservices must store the state and session data in a shared database such as Redis

42. What is CQRS pattern?

- **CQRS stands** for Command Query Responsibility Segregation
- Since each microservice has its own database, it is not possible to run SQL Queries across multiple tables in multiple domains
- To support querying a micro service will maintain a view of the data it needs to execute the query.
- To Keep the view database the microservice subscribes to domain events (Event Streaming)

43. Explain the API Gateway Pattern?

- Helps clients i.e., a Mobile App, connect to the micro services
- Without an API Gateway, clients get coupled with the microservices
- API Gateway can aggregate the API calls and reduce network roundtrips
- Security issues if no API Gateway
- API Gateway handles cross-cutting issues i.e., SSL, Auth, Access Log etc.

44. Explain Event Sourcing?

- Traditional Create, Read, Update and Delete (CRUD) has issues in micro services architecture.
  - CRUD is done on the online database, reducing performance because transactions lock the tables
  - In a micro services architecture multiple services may need to update a model (i.e., an order status) so concurrency is likely
  - It's not possible to know the history of events i.e., Status=1 WHY?

- **In Event Sourcing:**
  - Micro service subscribes to Events that are relevant

- Stores the events are they are received in an append-only table or database

45. What is Strangler Application?

- This is an approach for migrating legacy monolithic applications to Microservices
- Modernize an application by incrementally developing a new(strangler) application around the legacy application. In this scenario, the strangler application has a ***microservice architecture***.

46. What is API Composition?

- When each microservice has its own database, querying database with joins across two database or services is not possible.
- API composition queries multiple services and returns the result after joining them.

47. What are patterns of Cross Cutting Concerns?

Microservices Chassis: A framework handles Cross Cutting Concerns (CCC) i.e., Logging

- Spring boot
- Spring Cloud
- DropWizard
- Gizmo
- GoKit

Externalised Configuration: A service must run in multiple environments -dev, test, qa, staging, production – without modification and/or recompilation.

- Store database connection strings, log storage API endpoints etc in an external system in each environment i.e., AWS Config or Hashicorp Consul.

Service Template: Build a microservice template that includes all code for cross-cutting concerns. Use it to create new Microservices.

48. Explain Transactional Outbox Table?

- Microservices normally have to update a record locally and raise an event or send a message
- Sending a message in the middle of a transaction is not reliable
- Microservices add the message to a table while the transaction is in progress
- A message relay service reads the committed records from the event or message Outbox table.

49. What is Transactional Log Tailing?

- It's a pattern for reading events and messages from Outbox Table
- Guaranteed to be accurate
- Requires database-specific solution i.e., specific SQL Server.

50. What is Polling Publisher pattern?

- It’s a pattern for reading events and messages from Outbox Table
- Message Relay Services periodically queries the Outbox Table
- If new record found, Message Relay Service send them to Message Broker and then deletes the messages from the Outbox Table.
- Works with any SQL Database